The University of Hong Kong
School of Computing and Data Science

Interim Progress Report for COMP7705

Agentic-Quant: A Multi-Agent Framework for Reasoning-Based Quantitative Analysis

Mentor: Prof. Wu, Chuan

Group Members

Full Name	Student ID	Email
Ying Tingkai	3036657615	tkying2025@connect.hku.hk
Wang Wenhan	3036656398	u3665639@connect.hku.hk
Cao Yujuncheng	3036654819	U3665481@connect.hku.hk
Wang Pengcheng	3036656427	u3665642@connect.hku.hk
Gao Ziteng	3036654259	U3665425@connect.hku.hk

June 2026

Abstract

Since the March 2026 detailed project proposal, Agentic-Quant has undergone significant architectural evolution. The five-agent LangGraph pipeline remains the analytical core, but the system architecture has been rebuilt around a unified data layer and a modern React frontend, replacing the original Streamlit prototype.

Key achievements in this period include: (1) an automated multi-source data collection system that has accumulated 7,532 OHLCV records and 136 news articles across 24 tickers spanning US, Hong Kong, A-share, Japanese, and Korean markets; (2) a DataService abstraction layer that serves as the single entry point for all data access, eliminating code duplication and enabling consistent freshness management; (3) a Next.js 16 frontend with 8 of 15 pages connected to real APIs, featuring multi-currency support, real-time price display, and SSE-based streaming agent analysis; (4) a MonitorTask subsystem enabling keyword- and ticker-based continuous monitoring with optional AI-powered summarization; and (5) an OWM-weighted memory layer that records and recalls past trading decisions at decision time.

Several items from the original proposal — notably the Human-in-the-Loop approval mechanism and the broker mock engine — have been deferred in favor of more immediately impactful infrastructure: the data collection pipeline, unified data access layer, and continuous monitoring capabilities. These strategic pivots reflect a design philosophy that prioritizes robust data foundations over premature feature development.

Keywords: Multi-Agent System, LangGraph, Data Infrastructure, React Frontend, LLM-driven Analysis, Continuous Monitoring

1. Introduction

1.1. Background

When the detailed proposal was submitted in March 2026, Agentic-Quant had a working five-agent pipeline (market, news, fundamentals, risk, portfolio manager) running on LangGraph, a Streamlit-based web interface, and basic data retrieval from Yahoo Finance and Google News. The proposal outlined five enhancement directions: persistent context management, human-in-the-loop approval, broker simulation, multi-channel notification, and decision-flow visualization.

The subsequent three months of development have validated some of these directions while calling others into question. Most importantly, practical experience revealed that without a robust data infrastructure — automated collection, consistent storage, and unified access — all higher-level features would be built on an unreliable foundation.

1.2. What This Report Covers

This interim report documents the work completed between March and June 2026. Section 2 maps each proposal milestone against actual progress. Section 3 describes the system's architectural evolution, with emphasis on the data layer that emerged as the project's core contribution. Section 4 presents the React frontend now serving 8 pages with real data. Section 5 covers the Agent pipeline, memory system, and the newly added MonitorTask subsystem. Section 6 outlines strategic pivots from the original proposal. Section 7 lists deliverables and Section 8 presents the remaining roadmap.

2. Milestone Progress

The original proposal defined 10 milestones totaling 1,300 estimated hours. The table below maps each against actual progress.

#	Task	Proposed	Est. h	Status
1	Data pipeline (YFinance, Google News, AkShare)	2026-02	80	Completed — exceeded scope with APScheduler automation, FTS5 search, and DataCollector
2	Agent tools and LangGraph workflow	2026-03	120	Completed — 5-agent pipeline with tool binding, conditional routing, and SSE streaming
3	Streamlit web interface	2026-03	100	Migrated to Next.js 16 + React — 15 pages built, 8 connected to real APIs
Tasks 1–3 completed before proposal submission				
4	Context management and persistent storage (SQLite)	2026-04-14	150	Completed — ContextStore with per-strategy isolation, WAL mode, absolute paths, explicit commit model
5	Human-in-the-loop approval mechanism	2026-05-05	120	Deferred — market monitoring and data infrastructure prioritized; HITL requires broker engine first
6	Broker mock engine and backtesting	2026-06-01	200	Deferred — backtest logic designed but no broker layer; MonitorTask built as more practical alternative
7	Decision-flow visualization	2026-06-16	150	Not started
8	Multi-channel notification and integration	2026-07-06	160	Not started — Settings page has notification configuration UI; backend stubs exist
9	Project webpage and evaluation	2026-07-13	100	Not started
10	Final report and demo preparation	2026-07-17	120	In progress — this report
		Completed/adapted		Tasks 1–4, 450h
		Unplanned but delivered		DataService layer, React frontend, MonitorTask, Memory integration

2.1. Key Deviations Explained

Task 5 (HITL) and Task 6 (Broker) deferred. The original proposal envisioned a full trading loop: agents analyze, PM decides, human approves, broker executes. Practical development revealed that this loop requires market data, historical context, and performance tracking to be reliable **before** execution enters the picture. Without accumulated OHLCV records, news archives, and ticker metadata, both HITL and broker would operate on stale or incomplete data. The decision was made to build the data foundation first.

Unplanned additions. The React frontend, DataService abstraction, MonitorTask subsystem, and multi-market watchlist support emerged organically from the development process. Each addressed a concrete need discovered during implementation.

3. System Architecture

3.1. Architecture Overview

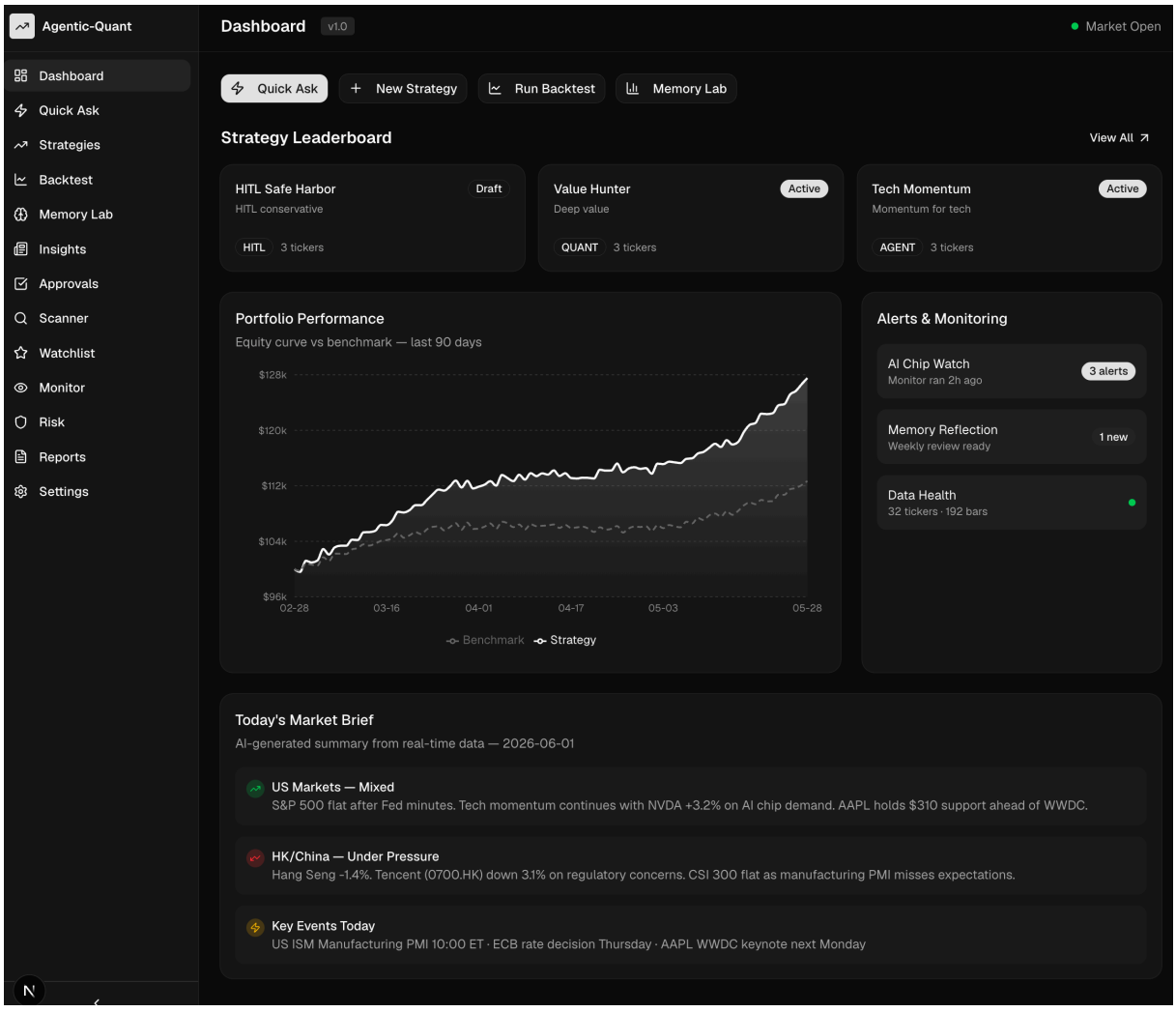

Figure 1: Dashboard page with strategy leaderboard cards showing live performance data.

The current system consists of four layers:

Layer	Components

Data Collection	APScheduler (5 background jobs), DataCollector (32 tickers), YFinance + AkShare + Google News providers, 7,532 OHLCV / 136 news accumulated
Data Access	DataService (single entry point: get_prices, get_news, get_meta, get_indicators, get_fundamentals, search_news), MarketDataStore, ContextStore, MemoryStore
Application	FastAPI server (38 API endpoints), 5-agent LangGraph pipeline, MonitorRunner, SSE streaming
Presentation	Next.js 16 + React frontend (8 pages with real APIs), TanStack Query, shadcn/ui, multi-currency Intl.NumberFormat

Key architectural change from the proposal: the Streamlit prototype has been replaced by a FastAPI backend with a decoupled React frontend. This enables typed API contracts, parallel development, and a richer user interface.

3.2. Data Layer: Unified DataService

The most significant architectural innovation since the proposal is the DataService abstraction. In the original design, each component (API endpoints, agent tools, data collector) accessed data through its own code path — some reading the database directly, others calling providers, others mixing both. This led to duplicated freshness logic, inconsistent error handling, and bugs that required fixes in multiple places.

The DataService consolidates all data access into six methods:

- `get_prices(ticker, start, end)` — returns complete date range, filling gaps from YFinance automatically. Cached hits return in under 20ms; cold misses trigger a live fetch (approximately 700ms) followed by database write.
- `get_news(ticker, window_days)` — searches by ticker column; triggers provider fetch (Google News → AkShare fallback) if the database is empty.
- `search_news(query, ticker?, limit)` — FTS5 full-text search over accumulated news content. Does not trigger live fetch.
- `get_fundamentals(ticker, date?)` — returns latest fundamentals snapshot; refreshes if older than 30 days.
- `get_meta(ticker)` — fetches company name, sector, currency, country, and exchange from YFinance once and caches permanently.
- `get_indicators(ticker)` — computes RSI, MACD, SMA, and ATR from stored OHLCV data; stores fetched data to enrich future cache hits.

Every consumer in the system — API endpoints, Agent tools, DataCollector, MonitorRunner, watchlist fetcher — now goes through these methods. Direct database access and direct provider calls have been eliminated from route-level code.

3.3. Multi-Source Data Collection

The DataCollector runs five APScheduler jobs within the FastAPI process:

Job	Provider	Interval
Price refresh	YFinance	15 min
News refresh	Google News → AkShare fallback	30 min
Sentiment refresh	Keyword-based aggregation	60 min
Macro calendar	Finnhub	Daily 08:00 UTC
Discovery pool	LLM-driven related ticker search	4 hours

The collector dynamically reads tickers from all user watchlists, merging them with a default pool. This ensures that any ticker a user adds to a watchlist automatically receives periodic data refresh. The system currently covers 32 active tickers across five markets.

3.4. Database Layout

Database	Contents
`market.db`	OHLCV (7,532 rows), fundamentals (58), news (136 + FTS5 index), ticker_meta (24), data_freshness
`system.db`	Strategies (3), watchlists (3), monitor_tasks (4)
`insights.db`	Monitoring reports (7)
`memory.db`	OWM-weighted trading memories (14)
`default.db`	Sessions (13), agent reports (36), decisions (5)

All databases use WAL journal mode with synchronous=NORMAL for concurrent read-write access.

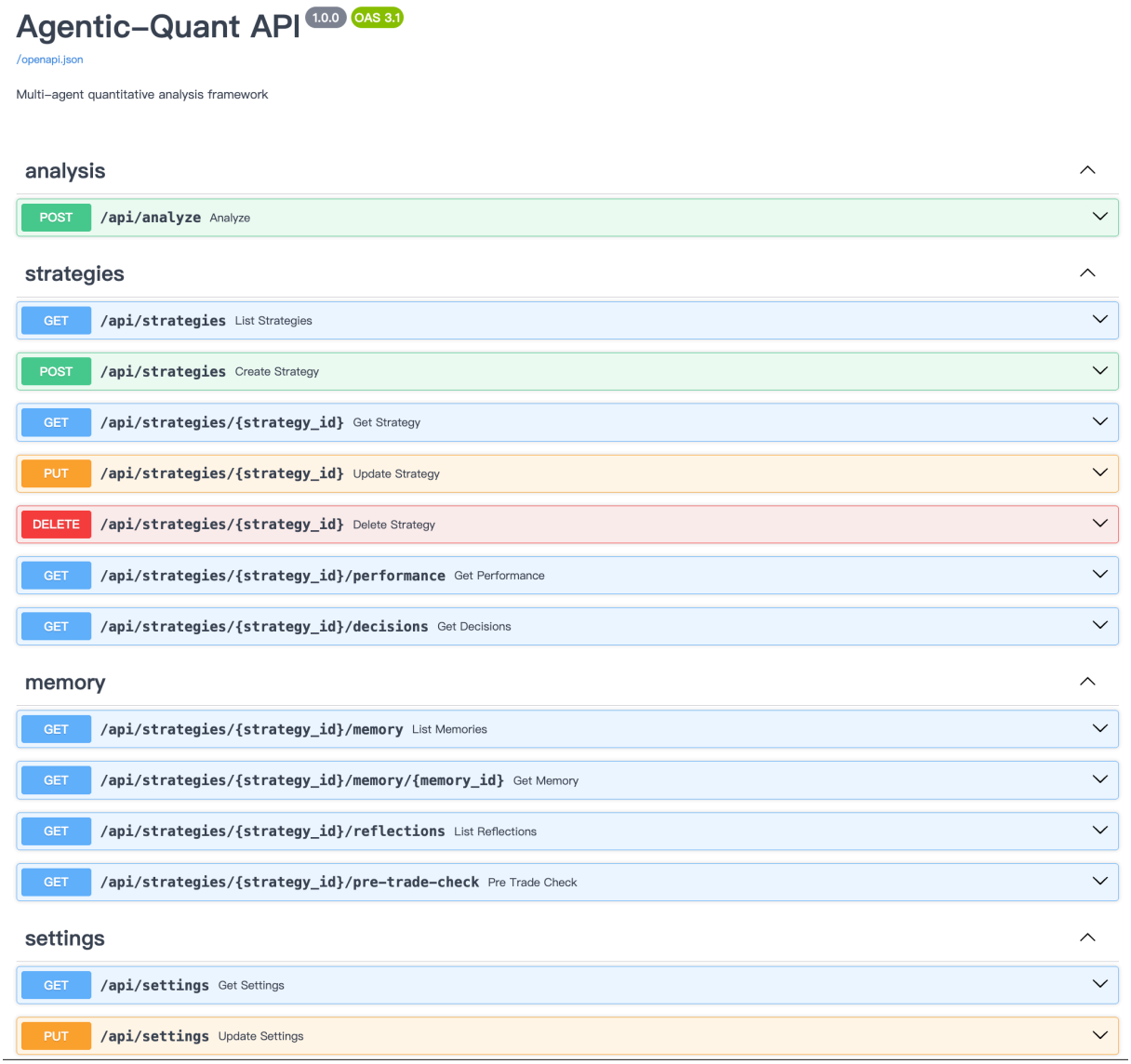

Figure 2: FastAPI Swagger documentation showing 38 implemented API endpoints.

Storage paths are resolved to absolute paths (derived from `__file__`), eliminating CWD-dependent data loss. Every write method explicitly calls `db.commit()`, fixing a silent data-loss bug discovered during testing.

4. Frontend: From Streamlit to React

4.1. Technology Migration

Category	Proposal (Streamlit)	Current (React)
Framework	Streamlit	Next.js 16 + React + TypeScript
UI Components	Streamlit widgets	shadcn/ui (30+ components)
Server State	st.cache_data	TanStack Query v5
Client State	st.session_state	Zustand

Category	Proposal (Streamlit)	Current (React)
Routing	Single page	App Router (15 routes)
Streaming	st.write_stream	SSE via fetch ReadableStream
Charts	Plotly	Recharts + TradingView Lightweight Charts
Styling	Streamlit themes	TailwindCSS v4

The Streamlit prototype served as a proof-of-concept but had fundamental limitations: no client-side routing, limited interactivity, poor performance with real-time data, and an inability to support the multi-page dashboard envisioned in the specification. The React migration addressed all of these.

4.2. Pages Connected to Real APIs

Page	Features	Data Source
Dashboard	Strategy cards with live performance metrics	`/api/strategies` + performance endpoints
Quick Ask	SSE-streaming agent analysis with debate display	`/api/analyze` (SSE)
Strategies	Full CRUD: create, list, detail, delete	`/api/strategies` (7 endpoints)
Watchlist	Multi-currency prices (USD / CNY / HKD / KRW), RSI(14), MACD, company names	`/api/market/*` + `/api/watchlists`
Memory Lab	OWM-weighted memory records with score distribution	`/api/strategies/:id/memory`
Monitor	Keyword/ticker monitoring, Run Now with search trace, report history	`/api/monitors` (7 endpoints)
Settings	LLM config, data source status, Test Connection	`/api/settings` + `/api/health`
New Strategy	Strategy creation form with type/ticker/belief configuration	`POST /api/strategies`

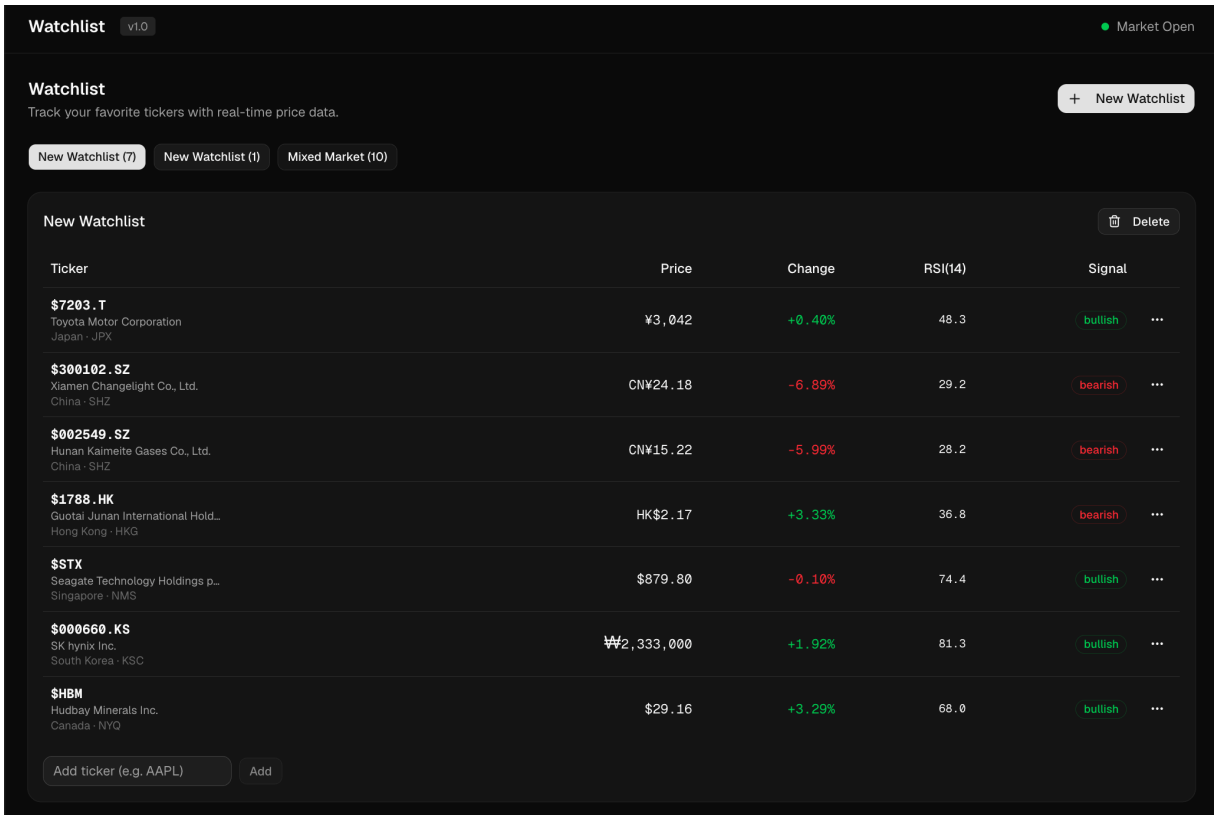

Figure 3: Watchlist page showing multi-market tickers with live prices and multi-currency support.

Eight of fifteen pages are now connected to real API endpoints delivering live data. The remaining seven pages (Scanner, Backtest, Risk, Approvals, Reports, Insights, and Conversation Detail) have functional UI shells that await backend implementations.

4.3. Watchlist: Multi-Market, Multi-Currency

The watchlist page exemplifies the frontend's capabilities. A user adds a ticker via the input field; the system immediately fetches OHLCV data from YFinance (if not already cached), stores company metadata (name, currency, country, exchange), then renders the price with the appropriate currency symbol. This is achieved through a single DataService call chain: `get_prices → get_meta → get_indicators`, all abstracted behind a `TickerRow` React component.

The `formatCurrency` utility delegates to `Intl.NumberFormat` with the ISO 4217 currency code returned by YFinance, eliminating hard-coded currency mappings. This means KRW, JPY, HKD, CNY, and any other currency work without additional configuration.

5. Agent Pipeline, Memory, and MonitorTask

5.1. Five-Agent Pipeline

The core analysis pipeline remains structurally unchanged from the proposal: market analyst, news analyst, and fundamentals analyst execute in parallel (each with their own tool set), feed into a risk analyst, which feeds into the portfolio manager for the final decision. The pipeline runs on LangGraph with conditional routing based on tool call detection. The architecture draws on established multi-agent frameworks in the literature [1], [2], [3].

Two significant additions have been made:

Memory recall before PM decision. Before the portfolio manager constructs its prompt, the system calls `MemoryStore.recall_by_context()` with the current ticker and market conditions. The top OWM-weighted memories are injected into the PM's system prompt as "relevant past decisions," providing the agent with historical context without requiring it to explicitly call a recall tool.

Memory remember after PM decision. A new graph node (`remember_memory`) has been inserted between the PM agent and the END node. After the PM produces its decision, this node constructs a `MemoryRecord` with the ticker, action, confidence, and report excerpt, computes its OWM score, and writes it to `memory.db`. This closes the observation → decision → memory loop, as advocated by systems like FinMem [4] and TradingAgents [1].

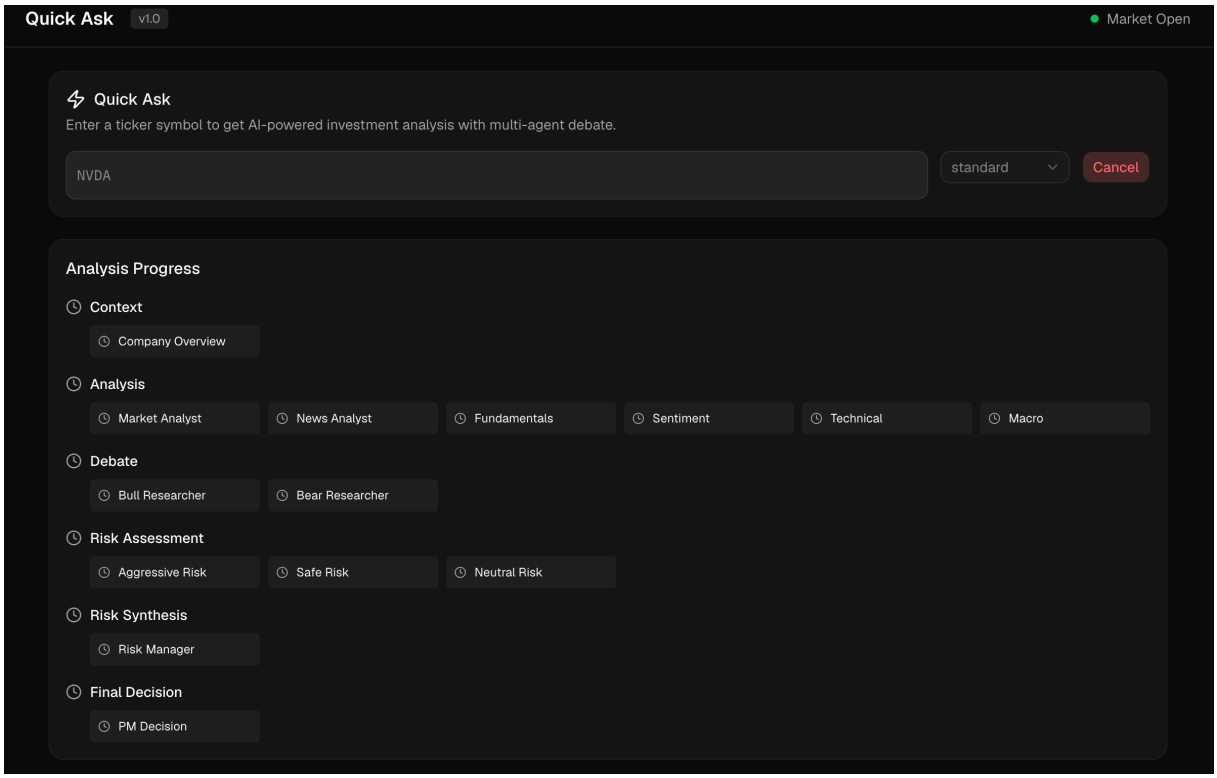

Figure 4: Quick Ask result: structured decision card with direction badge, confidence bar, timeframe, and one-liner summary.

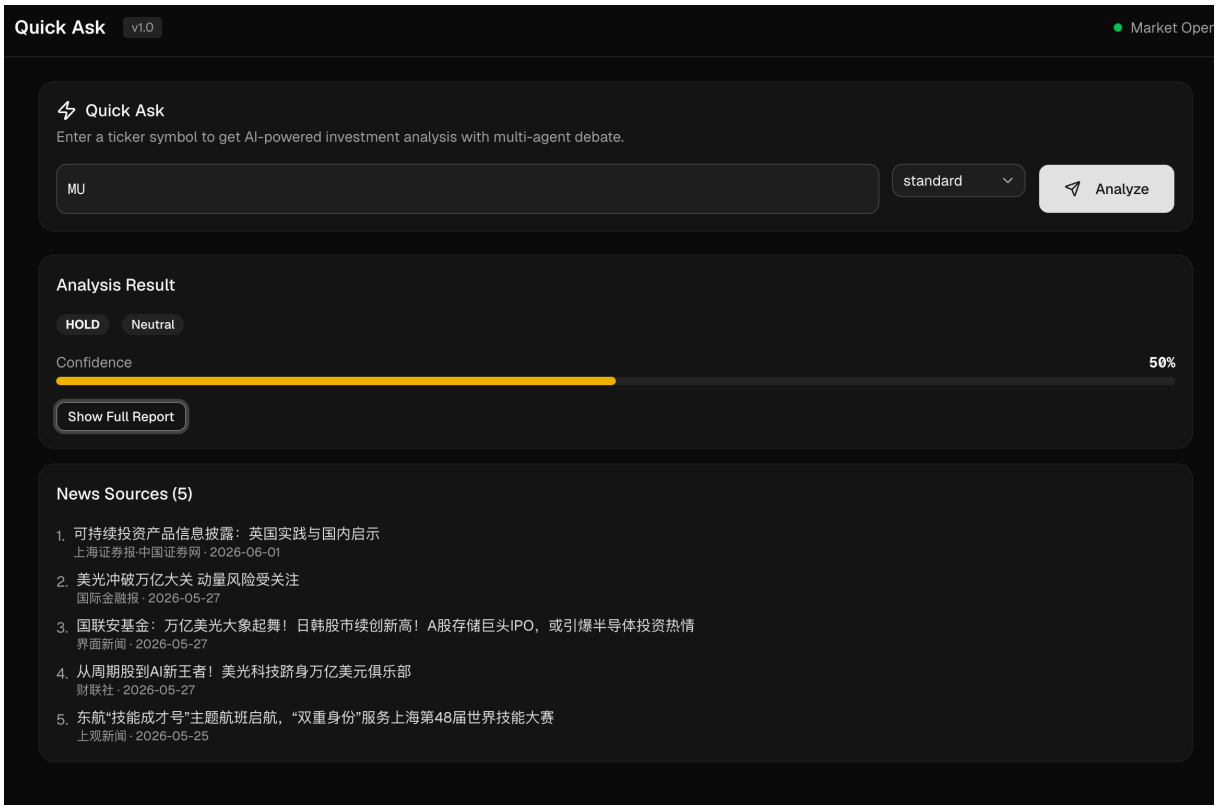

Figure 5: Quick Ask expanded full report with Markdown rendering and news source links.

5.2. Agent-Driven Data Discovery (Tier 4 Governance)

A discovery agent runs every 4 hours as a scheduled job. It reads the user's watchlist tickers and recent decision history, then calls the LLM with a constrained prompt to suggest 3–5 new tickers or topics worth tracking. Discovered tickers are added to a discovery pool for lazy data collection. This implements the "intelligent expansion" concept from the proposal's data governance discussion.

5.3. MonitorTask: Continuous Monitoring

The MonitorTask subsystem, not present in the original proposal, emerged as a more practical alternative to the broker engine. Instead of simulating trades, it provides continuous monitoring of keywords and tickers with structured reporting.

A MonitorTask is defined as: a monitoring mode (keyword or ticker), a set of targets (search terms or stock codes), a schedule (hourly/daily/weekly), and optional AI-powered summarization. A MonitorRunner background job checks active tasks every 5 minutes and executes those due according to their schedules.

Execution flow: the runner collects news via FTS5 search and price snapshots via DataService, then either generates a simple aggregate summary or calls the LLM to produce structured findings with sentiment analysis. Reports are stored in `insights.db` for historical retrieval. The frontend provides "Run Now" for immediate execution, a search trace showing exactly what was searched, and a report history view.

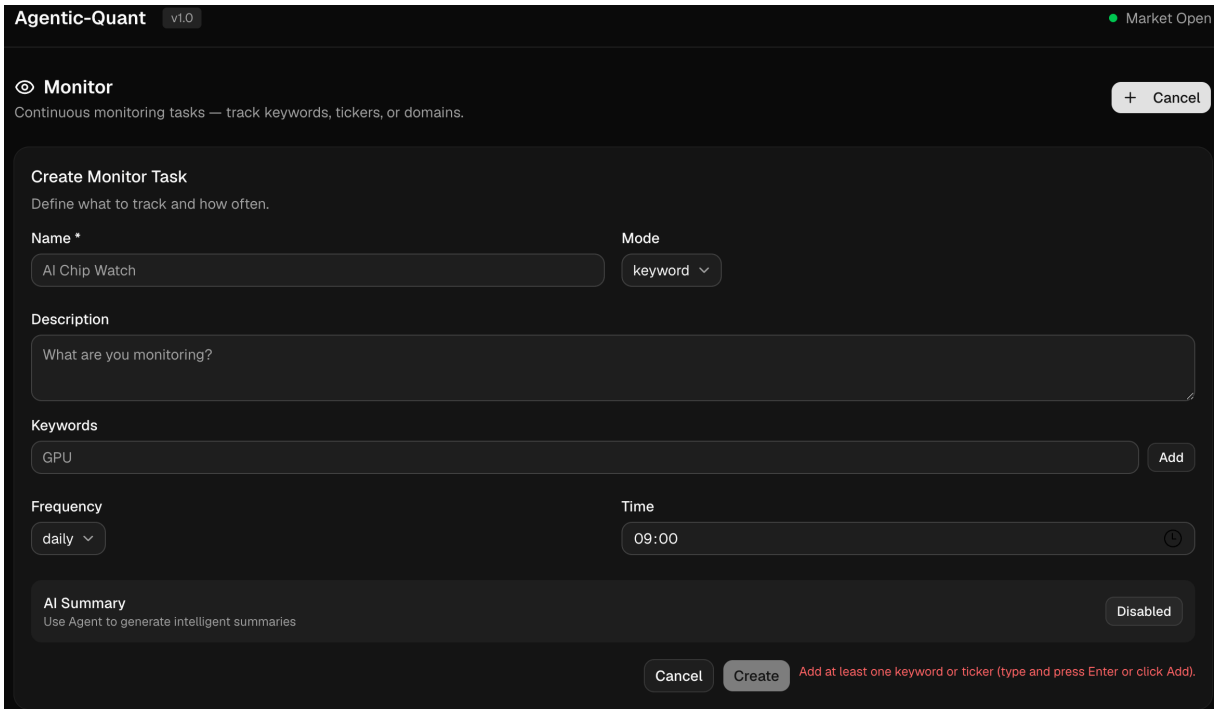

Figure 6: MonitorTask page: task list, creation form, Run Now execution, and report with search trace.

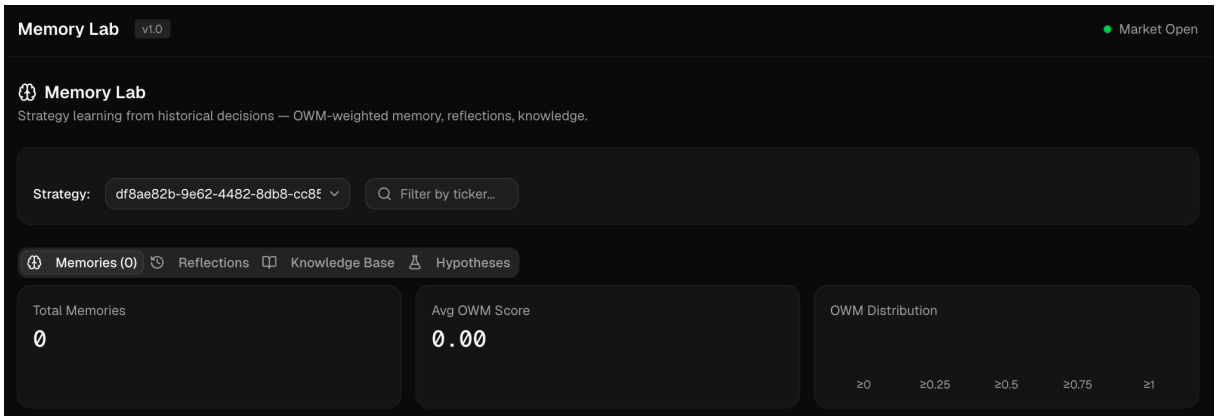

Figure 7: Memory Lab: OWM-weighted decision memories with score distribution visualization.

6. Strategic Pivots: What Changed and Why

6.1. From Streamlit to React + FastAPI

The proposal envisioned a Streamlit-based dashboard with decision-flow visualization. Streamlit worked well for the initial prototype but proved inadequate for a multi-page application: its single-page model cannot support the 15-page specification, its server-side rendering model conflicts with real-time data updates, and its widget library lacks the flexibility needed for financial dashboards.

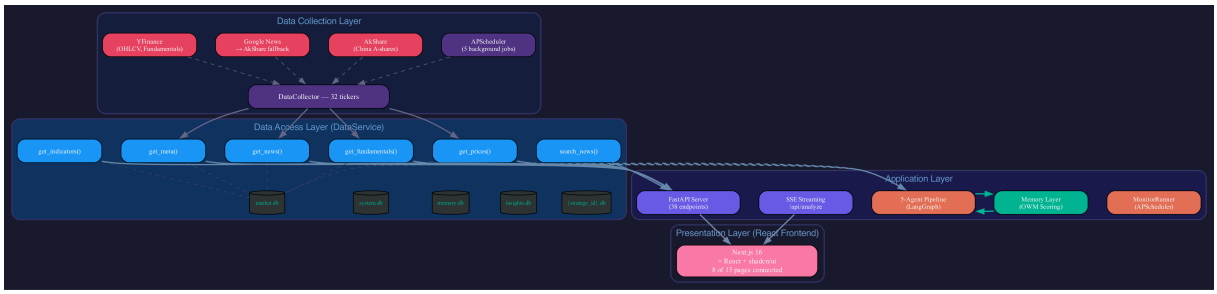

Figure 8: Current system architecture: four layers — Data Collection, DataService, Application, and React Frontend.

The migration to Next.js 16 + FastAPI separated concerns cleanly: the Python backend handles all data processing and agent orchestration; the React frontend handles all presentation. Communication is via typed REST APIs and SSE streaming. This architecture supports independent development of each layer and enables the rich interactive features visible in the current dashboard.

6.2. From HITL + Broker to Data Infrastructure + MonitorTask

The original proposal prioritized human-in-the-loop approval and broker simulation. Practical development revealed a more fundamental need: without accumulated market data (7,532 OHLCV records, 136 news articles) and a unified data access layer, neither HITL nor broker would function reliably. An approval decision based on stale prices or a trade executed against missing fundamentals would be worse than useless.

The strategic decision was to invest in data infrastructure first and to replace the speculative broker engine with the immediately useful MonitorTask. Monitoring — tracking keywords, tickers, and investment themes over time — provides tangible value without requiring a simulated exchange. It also aligns with the project's long-term vision of agent-driven research: the same News search and price retrieval infrastructure that powers MonitorTask today will power deep research pipelines tomorrow.

6.3. Why Data Architecture Matters

For a quantitative analysis system, data is not merely an input — it is the limiting factor. Without reliable, fresh, multi-source data, any agent output is speculation. The 7,532 accumulated OHLCV records and 136 news articles are not just statistics; they represent the systematic transformation of this project from a demo that queries APIs on-demand into a platform that builds knowledge over time.

7. Deliverables: Proposal vs. Current

Proposal Deliverable	**Status**	**Notes**
Working multi-agent system with persistent context	Delivered	ContextStore with 5 databases, WAL mode, explicit commits
Streamlit dashboard with decision-flow visualization	Replaced	React frontend replaces Streamlit; visualization deferred

Proposal Deliverable	Status	Notes
Multi-channel notification (WhatsApp/ Telegram)	Partial	Settings UI exists; backend stubs for email/telegram; no WhatsApp yet
Final report	In progress	This document
Demo video	Future	Planned for final submission

Unplanned deliverables produced: Working React frontend (8 pages with real data), DataService unified access layer, MonitorTask subsystem, OWM memory with recall/remember loop, multi-market watchlist with 5-currency support, agent-driven data discovery, FastAPI server with 38 API endpoints.

8. Next Steps

Based on the current state, the following priorities are identified for the remaining development period:

1. Strategy Execution. Strategies are currently configuration placeholders — they define tickers and schedules but never run automatically. The most impactful next feature is closing this loop: a scheduler-driven execution that runs the agent pipeline on each strategy's ticker list, records decisions, and enables performance tracking.

2. Deep Research Pipeline. A multi-stage analysis mode: script-driven parallel data collection (years of OHLCV, news archives, quarterly fundamentals), sub-agent analysis per dimension (technical, fundamental, news sentiment, industry), and a synthesis agent that produces a comprehensive research report. All intermediate outputs are permanently stored for reuse.

3. Backtesting Engine. Historical simulation using the accumulated OHLCV data: for each historical date in a selected range, run the agent pipeline with point-in-time data (no look-ahead), compare predicted directions against actual subsequent returns, and compute accuracy metrics against benchmarks. The 7,532 accumulated OHLCV records make this feasible without additional data collection. This addresses the common limitation identified in the backtesting literature [5], [6].

4. Settings Persistence. Currently the settings API returns an in-memory DEFAULT_CONFIG dictionary. Persisting settings to `system.db` will close the configuration loop.

9. References

The project continues to draw on the literature surveyed in the original proposal. Key references include:

References

[1] Y. Xiao, E. Sun, D. Luo, and W. Wang, "TradingAgents: Multi-Agents LLM Financial Trading Framework." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2412.20138

[2] Y. Yu *et al.*, "FinCon: A Synthesized LLM Multi-Agent System with Conceptual Verbal Reinforcement for Enhanced Financial Decision Making." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2407.06567

[3] H. Yang *et al.*, “FinRobot: An Open-Source AI Agent Platform for Financial Applications using Large Language Models.” Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2405.14767

[4] Y. Yu *et al.*, “FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design.” Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2311.13743

[5] H. Tatsat and A. Shater, “Beyond the Black Box: Interpretability of LLMs in Finance.” Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2505.24650

[6] A. de-la-Rica-Escudero, E. C. Garrido-Merchán, and M. Coronado-Vaca, “Explainable post hoc portfolio management financial policy of a Deep Reinforcement Learning agent,” *PLOS ONE*, vol. 20, no. 1, p. e315528, Jan. 2025, doi: 10.1371/journal.pone.0315528.